OS Command Injection

Study Notes: From Simple Injection to Blind Exploitation Techniques

Topics	OS command injection, blind injection, time delays, output redirection
Tooling	Burp Suite Community Edition, PortSwigger Web Security Academy
Labs covered	4 labs / exercises (2 Collaborator-based labs skipped)

Contents

- 1. Introduction to OS Command Injection
- 2. Lab 1 — Simple OS Command Injection
- 3. Blind OS Command Injection — Concepts
- 4. Lab 2 — Blind Injection with Time Delays
- 5. Lab 3 — Blind Injection with Output Redirection
- 6. Shell Operators — Quick Reference
- 7. Overall Key Takeaways

---

1. Introduction to OS Command Injection

What is OS Command Injection?

OS command injection happens when an application builds an operating system command using user input without safely handling it.

Example of vulnerable code:

```
system("ping " . $_GET["ip"]);
```

If the user enters `127.0.0.1`, the server executes:

```
ping 127.0.0.1
```

This is the developer's intended behavior.

How an Attacker Exploits It

Suppose the attacker enters `127.0.0.1; whoami`. The server now executes:

```
ping 127.0.0.1; whoami
```

Instead of executing one command, it executes two:

```
ping 127.0.0.1
whoami
```

The attacker has successfully made the server execute an OS command.

Why Command Injection Happens

- User input is inserted directly into an operating system command.
- The shell interprets special operators like `;`.
- The application does not properly validate or escape the input.

> In short: the application trusted user input inside a shell command.

SQL Injection vs. Command Injection

SQL Injection attacks the database — the attacker injects SQL into a database query.

```
SELECT * FROM users WHERE id='1'
```

Command Injection attacks the operating system — the attacker injects OS commands into a shell command.

```
ping 127.0.0.1; whoami
```

Linux Commands Learned

whoami

Returns the current operating system user. Used to prove command execution.

```
whoami
```

Output example:

```
www-data
```

or

```
peter
```

pwd

Print Working Directory — shows the current directory where the process is running.

```
pwd
```

Output example:

```
/var/www/html
```

Useful for understanding where the web application is executing.

2. Lab 1 — Simple OS Command Injection

Lab: OS command injection, simple case **Goal:** Execute `whoami` using command injection.

Understanding the Lab

The lab states that the application executes a shell command containing user-supplied product and store IDs. This means the server probably runs something conceptually like:

```
check_stock productId storeId
```

Example:

```
check_stock 3 1
```

We do not know the real command — `check_stock` is only a conceptual example used to understand how the application might work.

Identifying User-Controlled Inputs

The HTTP request contained:

```
productId=1
storeId=1
```

Both are user-controlled inputs. Either one could potentially be vulnerable.

Installing and Using Burp Suite

Burp Suite Community Edition was installed on the Windows host. Burp sits between the browser and the target website:

```
Browser -> Burp Suite -> Target Website
```

This allows interception and modification of HTTP requests before they reach the server.

Rather than configuring Chrome to proxy through Burp, Burp's built-in Chromium browser was used instead, since it is already configured to automatically send traffic through Burp with no proxy setup required.

Capturing the Request

Inside Burp: **Proxy** → **Intercept ON**, then clicked "Check stock". Burp intercepted:

```
POST /product/stock
```

Original request body:

```
productId=1&storeId=1
```

Payload Used

Modified request:

```
productId=1&storeId=1;whoami
```

Instead of only sending `storeId=1`, the application receives `storeId=1;whoami`. If inserted directly into a shell command, it becomes conceptually:

```
check_stock 1 1;whoami
```

Execution: (1) check stock, (2) execute `whoami`.

Why the Payload Worked

The application did not sanitize the user input. The shell interpreted `;` as "start another command," so `whoami` was executed.

Response Received

```
62
peter-oySx4p
```

- `62` — the stock checker still executed normally.
- `peter-oySx4p` — output of `whoami`, proving the server executed our injected OS command.

> Originally the application should only have returned "62". The only explanation for the extra line is that our modified request changed the shell command, the server executed our injected command, and its output was included in the response.

Why We Injected into storeId

We chose `storeId` because it is user-controlled — nothing special about it. We could also have tested `productId`; if one parameter fails, simply test the other. In real penetration testing, every user-controlled parameter is a potential attack surface until proven otherwise.

Overall Workflow Followed

- Read and understood the lab.
- Identified the user-controlled inputs (productId and storeId).
- Installed Burp Suite and used Burp Browser.
- Enabled Proxy Intercept and intercepted the stock-check request.
- Modified the request body, injecting `;whoami`.
- Forwarded the request and observed the server execute the injected command.
- Confirmed OS Command Injection through the returned username.

3. Blind OS Command Injection — Concepts

What is Blind OS Command Injection?

Blind OS command injection occurs when the injected command **does** execute on the server, but its output is **not** visible in the HTTP response.

Normal (visible) injection:

```
Injected: whoami
Response: peter-oySx4p
```

Blind injection:

```
Injected: whoami
Response: Thank you for your feedback.
```

No output is returned — but the command may still execute. You simply cannot see its output, so execution must be proven through **side effects**, such as:

- Response time delays
- Network requests / DNS lookups
- Ping requests
- File creation

Why Time Delays Work

If a normal request takes 0.5 seconds and an injected request consistently takes 10.5 seconds, and the only thing changed was your input, that ~10-second delay is strong evidence the injected command executed.

sleep 10

`sleep 10`

Pauses the current shell process for 10 seconds — it does not stop the server, only the shell process handling the request.

```
echo Start
sleep 10
echo End
```

Execution: `Start` → (wait 10 seconds) → `End`.

`sleep 10` produces no visible text, but it changes response time, and that visible delay proves execution — unlike `whoami`, whose output is useless if the application hides it.

Feedback Form Attack Surface

User-controlled inputs on the tested form: `name`, `email`, `subject`, `message`. The `csrf` token is different — it is server-generated and typically compared like:

```
if ($_POST["csrf"] == $_SESSION["csrf"])
```

It is not inserted into shell commands, so it is not a primary target for command injection.

Pentesting mindset: test, observe, and collect evidence — never assume one field is vulnerable, and never just guess.

One Variable at a Time

Keep every field normal except one being tested, e.g.:

```
name=test
email=test@test.com
subject=hello
message=<payload>
```

If multiple fields contain payloads and the attack works, you won't know which parameter caused it.

Quoting Concept

Suppose the application builds:

```
echo "hello"
```

If input becomes:

```
echo "hello;sleep 10"
```

then `;` is inside quotes and is treated as plain text, not a command separator.

> Understanding how input is incorporated into a command is more valuable than memorizing payloads.

Biggest Pentesting Lessons

- A failed payload does not prove there is no vulnerability — it only disproves one hypothesis.
- Do not memorize payloads; understand how shells interpret operators, how applications construct commands, and how user input is incorporated.
- Blind vulnerabilities are proven using observable behavior (time delays, network callbacks, DNS lookups, file creation), not visible output.
- Evidence is more important than assumptions. Keep asking: Did I test the correct parameter? The correct operator? Could the input be quoted or escaped? Is there another observable side effect?

4. Lab 2 — Blind Injection with Time Delays

Lab: Blind OS command injection with time delays **Goal:** Cause a 10-second delay in the server's response to prove an injected OS command executed (this lab does not return command output).

First Payload Attempt

Tried inside the `message` field:

```
hello;sleep 10
```

Response time: ≈1 second — no delay. A failed payload does not mean there is no vulnerability; it only means that particular hypothesis failed (wrong parameter, wrong operator, wrong shell syntax, input quoted/escaped/sanitized, etc.). A pentester does not stop after one failure.

Why We Tried `&&` Next

If the original command succeeds, `&&` should execute the payload immediately after it:

```
hello&&sleep 10
```

This also produced no delay. Neither payload proved the application was safe — they only proved those specific payloads did not execute. Responses still showed "Thank you for your feedback," meaning the application continued normally without crashing, suggesting the payload wasn't interpreted the way expected.

The Solution

The vulnerable parameter turned out to be **email**, not `message`. Applications frequently pass email addresses into shell commands used by `mail`, `sendmail`, `mailx`, or notification scripts — so the email field became part of the shell command, while the other fields in this lab did not.

Successful payload:

```
x||ping -c 10 127.0.0.1||
```

Why Start With x?

`x` is not a valid command, so it causes the first part of the shell command to fail. `||` means "if the previous command failed, execute the next one" — so `ping -c 10 127.0.0.1` runs.

Why ping?

```
ping -c 10 127.0.0.1
```

- `ping` — send ICMP echo requests
- `-c 10` — send exactly 10 requests
- `127.0.0.1` — localhost

Linux sends roughly one ping per second, so 10 pings ≈ 10 seconds. The shell waits until `ping` finishes before the HTTP response returns, producing the ≈10-second delay.

There is nothing wrong with sleep 10 as a command. The problem was how the application constructed the shell command — the assumptions behind ; and && simply didn't match the application's command structure, and || happened to work.

Why Didn't `&&` Work?

`command1 && command2` only runs `command2` if `command1` succeeds. In this lab, the first (injected) command did not succeed, so the payload after `&&` never executed.

Probable Server Logic (Conceptual)

We don't have the source code, but conceptually it may have looked like:

```
mail -s "Feedback" admin@company.com -f "<email>"
```

With the payload, execution conceptually becomes:

```
mail ... -f x||ping -c 10 127.0.0.1||
```

The first portion fails, and `||` triggers `ping`, which delays the response.

Key Takeaways

- Blind command injection hides command output; time delays can prove code execution.
- `sleep 10` (or `ping -c 10 127.0.0.1`) creates an observable side effect.
- Always test one parameter at a time.

- A failed payload does not mean the application is secure.
- `;` always executes the next command; `&&` only after success; `||` only after failure.
- This lab's vulnerable parameter was **email**; the successful payload was `x||ping -c 10 127.0.0.1||`.

5. Lab 3 — Blind Injection with Output Redirection

Lab: Blind OS command injection with output redirection **Goal:** The server executes the injected command but does not return its output in the HTTP response — however, it allows writing files into a publicly accessible folder, `/var/www/images/`, from which output can then be retrieved.

Initial Request

Intercepted feedback request body:

```
csrf=...
name=test
email=test@test.com
subject=hello
message=hello
```

The email field was already known to be vulnerable from previous labs.

Payload Used

The email value was replaced with:

```
test@test.com||whoami>/var/www/images/output.txt||
```

So the request became:

```
csrf=...
name=test
email=test@test.com||whoami>/var/www/images/output.txt||
subject=hello
message=hello
```

What the Payload Did

The injected part: `||whoami>/var/www/images/output.txt||`

- `whoami` — prints the username of the Linux account running the application (e.g. `peter-SQAkZX`)
- `>` — output redirection operator; instead of printing to the terminal, `whoami>/var/www/images/output.txt` saves the output into a file
- `/var/www/images/` — the lab stated this folder is writable and used to serve product images, so anything saved there becomes accessible from the website

Retrieving the Output

After submitting the request, visiting:

`/image?filename=output.txt`

returned:

`peter-SQAkZX`

— the output of `whoami`. Submitting that username solved the lab.

Why a Valid-Looking Email?

`test@test.com` was used because it is syntactically valid, satisfying the application's input validation — a clearly invalid email might be rejected before reaching the vulnerable code. Whether the email actually exists is irrelevant.

> Important clarification: the email address itself does not "execute." In something like `mail -f test@test.com`, `test@test.com` is only an argument passed to the real command — not a command itself.

Why `||` Worked (and `&&` Didn't)

`command1 || command2` runs `command2` only if `command1` fails. Conceptually:

```
mail -s "Feedback" admin@company.com -f test@test.com || whoami >/var/www/images/output.txt
```

The shell checks the **exit status of the mail command**, not whether the email address is valid. Since `&&` requires success and the original command apparently failed after injection, `&&` never executed the payload, while `||` did.

> Without the application's source code, the exact reason the underlying command failed is unknown. As penetration testers, it is enough to observe that && failed and || succeeded, and continue exploitation from there.

Collaborator-Based Labs (Skipped)

The remaining two OS command injection labs required **Burp Collaborator**, used to trigger an out-of-band DNS request, confirm execution when output can't be viewed or written to a file, and exfiltrate output (such as `whoami`) via DNS. These labs require Burp Suite Professional, since PortSwigger only allows interactions with its own public Collaborator server — Community Edition cannot generate or poll Collaborator payloads, so these two labs were skipped.

Key Concepts Learned

- Blind OS command injection means commands execute but their output is not shown.
- Output redirection (>) can save command output into a file.
- If that file is web-accessible, it can be retrieved through the application.
- `whoami` identifies the OS user running the application.

- A valid-looking email is often used to satisfy application validation — it is an argument, not the command.
- `||` checks the exit status of the previous command, not the validity of the input; `&&` runs only after success, `||` only after failure.
- Without source code, the exact reason for a failure is unknown — exploit development relies on observed behavior rather than assumptions.

6. Shell Operators — Quick Reference

Operator	Meaning	Example	Notes
;	Always run the next command, regardless of the first command's success.	`ping 127.0.0.1; whoami`	Most common operator for simple command injection.
&&	Run the next command only if the first command succeeds.	`ping google.com && whoami`	Fails silently if the first command fails.
||	Run the next command only if the first command fails.	`x||ping -c 10 127.0.0.1||`	Useful when the first (deliberately invalid) command is expected to fail.
|	Pipe: pass the output of one command as input to another.	`cat file.txt | grep admin`	Connects commands rather than simply running an independent second one.
>	Redirect a command's output into a file instead of the terminal.	`whoami>/var/www/images/output.txt`	Useful for blind injection when the output file is web-accessible.

Linux Commands Referenced

- **whoami** — returns the current OS user; used to verify command execution.
- **pwd** — print working directory; shows where the process is executing.
- **sleep N** — pauses the shell process for N seconds; used to create an observable delay in blind injection.

- **ping -c N host** — sends N ICMP echo requests (~1 per second); often used instead of sleep to create a measurable delay.

7. Overall Key Takeaways

- OS command injection occurs when user input is inserted into a shell command without proper validation or escaping.
- The shell interprets special operators (`;`, `&&`, `||`, `|`) — understanding their exact behavior matters more than memorizing payloads.
- **Visible** injection is confirmed by seeing command output (e.g. `whoami`) directly in the response.
- **Blind** injection must be confirmed through observable side effects: time delays, file writes to web-accessible paths, or out-of-band network/DNS interactions.
- Always test one user-controlled parameter at a time, so a working payload can be attributed to the correct field.
- A failed payload disproves only one hypothesis — not the presence of a vulnerability. Keep testing different operators and parameters.
- Input may be quoted, escaped, or sanitized in ways that block one operator (e.g. `;`) while leaving another (e.g. `||`) exploitable — this depends entirely on how the application builds its shell command, which is invisible without source code.
- Burp Suite (Community Edition, with its built-in browser) is the primary tool for intercepting, modifying, and replaying HTTP requests during this kind of testing.
- Advanced blind-injection scenarios with no visible output and no writable web-accessible path require out-of-band techniques (e.g. Burp Collaborator DNS interactions), which need Burp Suite Professional.